

Ghost Gutzwiller Approximation for the Half-Filled Hubbard Dimer: Equations, Conventions, and Numerical Implementation

gGut_mau project — gga_dimer.py

September 16, 2026

Contents

1	Physical model	2
2	The ghost Gutzwiller ansatz: general framework	2
3	Theory: the constrained variational Lagrangian	3
3.1	Euler–Lagrange equations	5
4	General workflow: inputs, guesses, and order of computation	6
5	Specialization to the two-site, two-independent-fragment dimer	8
5.1	Each site is its own fragment; the two fragments need not be equal	8
5.2	Quasiparticle Hamiltonian for two independent fragments	8
5.3	Filling rule (half filling)	8
6	Self-consistency equations	9
6.1	Hybridization V (Eq. 7 of the reference)	9
6.2	Bath potential λ^c (Eq. 8 of the reference)	10
6.3	Impurity (embedding) Hamiltonian	11
6.4	Update of R and Δ (Eq. 10 of the reference)	13
6.5	Variational energy	14
7	Numerical solution of the self-consistency loop	14
7.1	Determining λ^0, λ^1 : a coupled, non-unique root-finding problem	14
7.2	Analytic Jacobian of the quasiparticle projector	16
7.3	Outer loop: mixing and convergence	17
7.4	The main self-consistency loop, in pseudocode	18
7.5	An abandoned fix: gauge-fixing	19
8	Observables reported by the code	19
9	Post-processing: quasiparticle Green’s function and spectral function	20
10	Command-line interface and input file formats	21

Purpose of this note. This document is a pedantic, self-contained description of *exactly* what the code in `gga_dimer.py` computes: every equation, every index convention, every regularization constant, and every numerical safeguard, together with the reasoning behind each choice. It is meant to let a reader reproduce the code from the equations alone, or verify the code against the equations line by line. Physical background (the ghost Gutzwiller approximation, gGA) is kept to the minimum needed to fix notation; references to the original literature are given in Section 12.

1 Physical model

We study the two-site (“dimer”) single-band Hubbard Hamiltonian

$$H_{\text{phys}} = -t \sum_{\sigma=\uparrow,\downarrow} \left(c_{0\sigma}^\dagger c_{1\sigma} + c_{1\sigma}^\dagger c_{0\sigma} \right) + U \sum_{i=0,1} n_{i\uparrow} n_{i\downarrow} - \mu \sum_{i=0,1} (n_{i\uparrow} + n_{i\downarrow}), \quad (1)$$

with sites labeled $i = 0, 1$, one physical orbital per site ($N_{\text{phys}} = 1$), and both spins present. The hopping amplitude t sets the energy unit; the code always uses $t = 1$ unless stated otherwise (the CLI flag `--t` allows changing it, and every energy reported by the code is implicitly in units of t).

Half filling and particle-hole symmetry. The chemical potential is fixed to the particle-hole symmetric value

$$\mu = U/2, \quad (2)$$

which pins the total occupation to $\langle n_{0\sigma} + n_{0\bar{\sigma}} + n_{1\sigma} + n_{1\bar{\sigma}} \rangle = 2$ (one electron per spin, summed over both sites) for *any* U . This is the only filling considered by the code; there is no doping/chemical-potential scan.

Exact benchmark. `exact_dimer.py` diagonalizes Eq. (1) exactly in the full $2^4 = 16$ -dimensional Fock space of the four physical modes ($i \in \{0, 1\}$, $\sigma \in \{\uparrow, \downarrow\}$), by building the four fermionic annihilation operators via an explicit Jordan–Wigner string (see Section 6.3 for the general construction, which is shared with the impurity solver). It reports the ground-state energy E_0 , total filling $\langle n \rangle$, and the double occupancy per site $\text{docc} = \frac{1}{2} \sum_i \langle n_{i\uparrow} n_{i\downarrow} \rangle$, used throughout as the ground truth that the gGA approximation is checked against.

2 The ghost Gutzwiller ansatz: general framework

The ghost Gutzwiller approximation (gGA) approximates the ground state of Eq. (1) variationally as $|\Psi_G\rangle = P|\Psi_{\text{qp}}\rangle$, where $|\Psi_{\text{qp}}\rangle$ is the Slater-determinant ground state of an auxiliary, purely quadratic *quasiparticle* Hamiltonian H_{qp} , and P is a Gutzwiller-type projector onto the physical Hilbert space. In the standard (“rotationally invariant slave boson”/gGA) formulation, the projector is replaced by an equivalent *embedding* (impurity) problem, and the whole scheme reduces to a self-consistency loop between:

1. a one-body quasiparticle Hamiltonian $H_{\text{qp}}(R, \lambda)$, built from a *renormalization matrix* R and an *onsite potential* λ , and

2. a many-body *impurity* Hamiltonian $H_{\text{imp}}(V, \lambda^c)$, describing one physical (correlated) orbital coupled to N_{eff} auxiliary (“ghost”) bath orbitals through a hybridization V and a bath-onsite potential λ^c .

Dimension bookkeeping. Let N_{phys} be the number of physical orbitals per fragment (here $N_{\text{phys}} = 1$: one site, one orbital, spin handled symmetrically) and let N_g be the number of *extra* ghost bath orbitals requested by the user, on top of the single bath orbital that even the *non-ghost* (ordinary) Gutzwiller/slave-boson theory already has. Define

$$N_{\text{eff}} = N_{\text{phys}} + N_g = 1 + N_g. \quad (3)$$

$N_g = 0$ is *not* “no bath”; it is the ordinary single-bath Gutzwiller theory. Each fragment’s embedding Hamiltonian then has $N_{\text{phys}} + N_{\text{eff}} = 2 + N_g$ orbitals per spin (one physical, N_{eff} bath), so, including both spins, the impurity Fock space has dimension

$$\dim(\mathcal{F}_{\text{imp}}) = 2^{2(1+N_{\text{eff}})} = 4^{1+N_{\text{eff}}} = 4^{2+N_g}. \quad (4)$$

(An earlier draft of the project notes stated 4^{1+N_g} ; that is off by one power of 4 and has been corrected in this document and cross-checked against the code, which builds exactly $2(1 + N_{\text{eff}})$ fermionic modes – see Section 6.3.)

3 Theory: the constrained variational Lagrangian

The self-consistency equations used throughout this document (Eqs. (14), (18), (33) in Section 6) are not postulated independently: they are the stationarity (Euler–Lagrange) conditions of a single constrained variational functional – a Lagrangian – whose extremization is equivalent to minimizing $\langle \Psi_G | H_{\text{phys}} | \Psi_G \rangle$ over the Gutzwiller-projected variational family of Section 2. **Five**, not four, independent Lagrange multipliers appear in this Lagrangian – it is essential to keep all five, because two of them (E_{qp} and $E^{I,c}$) are what *turn out*, only after the Euler–Lagrange equations are taken, to equal the quasiparticle and embedding ground-state energies; they are not postulated as energies from the start. This section reproduces the exact Lagrangian derived in `NotesOngGut.pdf` (its Eq. 29, itself the finite-embedding rewrite of the periodic-lattice Lagrangian, Eq. 19, once the projector matrix ϕ is traded for an explicit embedding wavefunction $|\Phi^I\rangle$ – see also Mejuto-Zaera, arXiv:2403.05157, Eqs. 5–6), specialized here to our two real-space fragments (no Brillouin-zone sum, and the two fragments *not* assumed equal, per Section 5). The precise algebraic form of the resulting equations (fixed by the exact operator-ordering convention already used in H_{imp} , Eq. (22), and cross-checked term by term against the independent reference implementation) is worked out operationally in Section 6.

Variational objects. For each fragment $I \in \{0, 1\}$:

- $|\Psi_{\text{qp}}\rangle$: the many-body quasiparticle wavefunction – a genuinely independent variational object, *not* yet assumed to be any particular eigenstate of any particular Hamiltonian. (It is only the Euler–Lagrange equation below that forces it to be an eigenstate of H_{qp} , and only the physical half-filling requirement, Section 5.3, that then picks out the filled Fermi sea, Eq. (12), as the relevant one.)
- $\Delta^{II} \in \mathbb{R}^{N_{\text{eff}} \times N_{\text{eff}}}$: the *shared* target one-body density matrix of the N_{eff} ghost/bath orbitals – the single object that both $|\Psi_{\text{qp}}\rangle$ and $|\Phi^I\rangle$ below are separately constrained to reproduce (this

is exactly the `Delta00.target/Delta11.target` variable that the outer loop of Section 7.3 iterates on);

- $R^I \in \mathbb{R}^{N_{\text{eff}}}$: the renormalization vector entering H_{qp} , Eq. (11);
- $|\Phi^I\rangle$: the many-body embedding (impurity) wavefunction – likewise independent until its own Euler–Lagrange equation below forces it to be an eigenstate of H_{imp} ;
- $E_{\text{qp}}, E^{I,c}$: Lagrange multipliers enforcing, respectively, $\langle \Psi_{\text{qp}} | \Psi_{\text{qp}} \rangle = 1$ and $\langle \Phi^I | \Phi^I \rangle = 1$;
- $\lambda^I, \lambda^{I,c} \in \mathbb{R}^{N_{\text{eff}} \times N_{\text{eff}}}$, $V^I \in \mathbb{R}^{N_{\text{eff}}}$: Lagrange multipliers described below, which *after* extremization take on the roles of the onsite qp potential, the bath potential, and the hybridization entering $H_{\text{qp}}/H_{\text{imp}}$.

The Lagrangian. Adapting Eq. 29 of `Notes0ngGut.pdf` to two real-space fragments coupled by hopping $-t$ (in place of a Brillouin-zone sum over a translationally invariant lattice):

$$\begin{aligned}
L = & -t \sum_{I \neq J} (R^I)^T \langle \Psi_{\text{qp}} | d_I^\dagger d_J | \Psi_{\text{qp}} \rangle R^J + \sum_I \langle \Phi^I | H_{\text{loc}}^I | \Phi^I \rangle \\
& + E_{\text{qp}} (1 - \langle \Psi_{\text{qp}} | \Psi_{\text{qp}} \rangle) + \sum_I E^{I,c} (1 - \langle \Phi^I | \Phi^I \rangle) \\
& + \sum_I \left\{ (V^I)^T \left[\langle \Phi^I | d_I^\dagger c_I | \Phi^I \rangle - \sqrt{\Delta^{II}(\mathbf{I} - \Delta^{II})} R^I \right] + \text{h.c.} \right\} \\
& + \sum_I \text{Tr} \left[\lambda^{I,c} (\Delta^{II} - \langle \Phi^I | d_I d_I^\dagger | \Phi^I \rangle) \right] + \sum_I \text{Tr} \left[\lambda^I (\Delta^{II} - \langle \Psi_{\text{qp}} | d_I^\dagger d_I | \Psi_{\text{qp}} \rangle) \right], \quad (5)
\end{aligned}$$

where $\mathbf{I}$ is the $N_{\text{eff}} \times N_{\text{eff}}$ identity matrix and, exactly as in H_{imp} (Eq. (22)), the $\lambda^{I,c}$ -term is written with the operator ordering $d_I d_I^\dagger$ (annihilation before creation), not $d_I^\dagger d_I$ – the two differ by the anticommutator identity of Eq. (25): using the “wrong” ordering here would reintroduce the sign bug already fixed in H_{imp} itself.

Role of each term.

1. *Kinetic term.* The bare quasiparticle hopping energy, evaluated on the as-yet-unconstrained wavefunction $|\Psi_{\text{qp}}\rangle$. Once the Euler–Lagrange equation below is imposed, its expectation value becomes exactly the $-4t (R^0)^T \Delta^{01} R^1$ kinetic contribution to the final variational-energy formula, Eq. (36).
2. $\sum_I \langle \Phi^I | H_{\text{loc}}^I | \Phi^I \rangle$. The physical interaction-plus-chemical-potential energy of each fragment’s embedding wavefunction. Together with the kinetic term, and once every constraint below is satisfied, these give $E_{\text{var}} = E_{\text{loc}}^0 + E_{\text{loc}}^1 - 4t (R^0)^T \Delta^{01} R^1$, Eq. (36).
3. E_{qp} . Enforces normalization of $|\Psi_{\text{qp}}\rangle$. Section 3.1 shows that extremizing L over $\langle \Psi_{\text{qp}} |$ turns this multiplier into exactly the quasiparticle ground-state energy of H_{qp} .
4. $E^{I,c}$. Enforces normalization of $|\Phi^I\rangle$. Extremizing L over $\langle \Phi^I |$ (Section 3.1) turns this multiplier into exactly the embedding ground-state energy E_{loc}^I that `impurity.solve` returns – unlike an earlier draft of this document, this multiplier is *not* inert: it is the eigenvalue of H_{imp}^I used throughout Section 6.3.

5. V^I -term. Enforces that the hybridization density $\langle \Phi^I | d_I^\dagger c_I | \Phi^I \rangle$ actually produced by the embedding wavefunction equals the qp-side amplitude $\sqrt{\Delta^{II}(\mathbf{I} - \Delta^{II})} R^I$ (the matrix square root converting a bath *occupation* into a coherent single-particle *amplitude*). This is the same relation as Eq. 16–17 of `NotesOngGut.pdf`. Stationarity with respect to V^I itself simply reinstates this constraint, which *is* the R -update, Eq. (33), of Section 6.4 (with $D^I = \langle \Phi^I | d_I^\dagger c_I | \Phi^I \rangle$); after extremization, V^I itself becomes the hybridization entering H_{imp} .
6. $\lambda^{I,c}$ -term. Enforces that the bath sector of the embedding wavefunction equals the shared variable Δ^{II} ; after extremization, $\lambda^{I,c}$ becomes exactly the bath potential appearing in H_{imp} , Eq. (22).
7. λ^I -term. Enforces that the 1-RDM block of the *quasiparticle* wavefunction, $\langle \Psi_{\text{qp}} | d_I^\dagger d_I | \Psi_{\text{qp}} \rangle$, also equals the same shared variable Δ^{II} ; after extremization, λ^I becomes exactly the onsite potential entering H_{qp} , Eq. (11). This is the fifth multiplier: it is easy to miss because, operationally (Section 7.1), the code solves for λ^I *directly* by demanding that $H_{\text{qp}}(R, \lambda^I)$'s own filled-Fermi-sea 1-RDM equal $\Delta_{\text{target}}^{II}$ – which is precisely this constraint, just solved for λ^I instead of carried as a separate symbol.

3.1 Euler–Lagrange equations

Variation with respect to $\langle \Psi_{\text{qp}} |$. Only the kinetic term and the λ^I -term depend on $|\Psi_{\text{qp}}\rangle$; setting $\delta L / \delta \langle \Psi_{\text{qp}} | = 0$ gives

$$\left[-t \sum_{I \neq J} R^I (R^J)^T d_I^\dagger d_J - \sum_I \lambda^I d_I^\dagger d_I \right] |\Psi_{\text{qp}}\rangle = E_{\text{qp}} |\Psi_{\text{qp}}\rangle, \quad \text{i.e.} \quad H_{\text{qp}} |\Psi_{\text{qp}}\rangle = E_{\text{qp}} |\Psi_{\text{qp}}\rangle, \quad (6)$$

recovering exactly the two-fragment quasiparticle Hamiltonian, Eq. (11): λ^I is confirmed to be a genuine onsite potential inside H_{qp} , and E_{qp} is confirmed to be its ground-state energy.

Variation with respect to $\langle \Phi^I |$. Only $\langle \Phi^I | H_{\text{loc}}^I | \Phi^I \rangle$, the V^I -term, and the $\lambda^{I,c}$ -term depend on $|\Phi^I\rangle$; setting $\delta L / \delta \langle \Phi^I | = 0$ gives

$$\left[H_{\text{loc}}^I + \sum_a V_a^I (d_{I,a}^\dagger c_I + c_I^\dagger d_{I,a}) - \sum_{ab} \lambda_{ab}^{I,c} d_{I,b} d_{I,a}^\dagger \right] |\Phi^I\rangle = E^{I,c} |\Phi^I\rangle, \quad \text{i.e.} \quad H_{\text{imp}}^I |\Phi^I\rangle = E_{\text{loc}}^I |\Phi^I\rangle, \quad (7)$$

recovering exactly the embedding Hamiltonian and its sign convention, both already fixed and validated in Eq. (22)/Eq. (25).

Variation with respect to R^I . R^I appears both in the kinetic term (as a coefficient of $\Delta^{IJ,(\text{qp})} \equiv \langle \Psi_{\text{qp}} | d_I^\dagger d_J | \Psi_{\text{qp}} \rangle$) and in the V^I -term; setting $\partial L / \partial R^I = 0$ gives

$$-2t \Delta^{IJ,(\text{qp})} R^J - \sqrt{\Delta^{II}(\mathbf{I} - \Delta^{II})} V^I = 0 \implies \sqrt{\Delta^{II}(\mathbf{I} - \Delta^{II})} V^I = -2t \Delta^{IJ} R^J, \quad (8)$$

which is precisely the structure of the V -fit, Eq. (14) (the overall numerical prefactor of that equation, $-t$ rather than $-2t$, is a convention choice tied to how the fragment sum and the h.c. in Eq. (5) are counted, and is fixed, independently of this derivation, by the term-by-term validation against the reference implementation described in Section 6.1). This – not the V^I -term's own stationarity – is where the V -fit equation comes from; varying with respect to V^I itself, as noted above, gives back the R -update, Eq. (33), instead.

Where Eq. (18) comes from. Δ^{II} enters only the V^I -term, the $\lambda^{I,c}$ -term, and the λ^I -term. Setting $\partial L / \partial \Delta^{II} = 0$, holding R^I, V^I *fixed* (i.e. without first re-expressing V^I through Eq. (14) as a function of Δ^{II}), gives

$$\lambda^I + \lambda^{I,c} = \left. \frac{\partial}{\partial \Delta^{II}} \left[(R^I)^T \sqrt{\Delta^{II}(\mathbf{I} - \Delta^{II})} V^I \right] \right|_{\text{sym}}, \quad (9)$$

which is precisely Eq. (18) – the origin of the statement, made in Section 6.2, that λ^I and $\lambda^{I,c}$ are not independent variational parameters but two of the five Lagrange multipliers of Eq. (5), tied together by this single stationarity condition. Stationarity with respect to $\lambda^I, \lambda^{I,c}$ themselves simply reinstates the two density-matching constraints they were introduced to enforce.

4 General workflow: inputs, guesses, and order of computation

Sections 6 and 7 give the full, precise algebraic form of every equation the code evaluates. Before diving into that detail, this section is a bird’s-eye map of the algorithm: what is *fixed* for the whole run, what is only ever a *guess*, and the exact *order* in which every other quantity is computed from those two. Every box below names the equation or section where it is defined in full; read this section first, then Sections 5–7 for the details it glosses over.

Three categories of quantity. Every symbol that appears anywhere in this document falls into exactly one of three categories:

1. **Fixed inputs** – set once, from the command line, and never changed by the algorithm itself: U, N_g (hence $N_{\text{eff}} = 1 + N_g$), t , the convergence tolerance tol , the maximum iteration count, and the mixing parameter mix (Section 7.3).
2. **A guess** – has a value only because *something* has to start the iteration; it is never computed from a formula that is trusted to be exact on its first use. There is exactly one such object: the pair (R^I, λ^I) at iteration 0, coming either from `R.in/L.in` or from the default cold start (Section 7.3). Every later “guess” used inside the λ -fit (Section 7.1) is itself *derived* (Eq. (38)) from quantities already computed in the *previous* iteration – it is a good starting point for a local solver, not an independent free parameter.
3. **Derived quantities** – everything else ($\Delta^{II}, \Delta^{IJ}, V^I, \lambda^{I,c}, |\Phi^I\rangle, E_{\text{loc}}^I, D^I$, the updated R^I, Δ^{II} , and finally E_{var}) is computed, in a fixed order, from the current (R^I, λ^I) and the previous iteration’s $(V^I, \lambda^{I,c})$. This order is the content of the flowchart below.

Order of computation, one outer iteration.

1. Diagonalize the quasiparticle Hamiltonian $H_{\text{qp}}(R, \lambda)$ and fill its lowest N_{eff} levels per spin to get Δ^{II}, Δ^{IJ} (Section 5.3).
2. Fit the hybridization V^I from R^I, Δ (Eq. (14)).
3. Compute the bath potential $\lambda^{I,c}$ from Δ, R^I, V^I and the *current* λ^I (Eq. (18)).
4. Diagonalize the embedding Hamiltonian $H_{\text{imp}}(V, \lambda^c)$ to get $|\Phi^I\rangle$, its energy E_{loc}^I , and the 1-RDM blocks $D^I, \Delta^{I,\text{bb}}$ (Section 6.3).
5. Update $\Delta^{II} \leftarrow \mathbf{I} - \Delta^{I,\text{bb}}$ and refit R^I from D^I, Δ^{II} (Eq. (33)).

6. Compare the updated (R^I, Δ^{II}) against their values at the start of this iteration. If the change is below tol , stop and report E_{var} (Eq. (36)) and the other observables of Section 6.5.
7. Otherwise, form a new λ^I guess from this iteration's $V^I, \lambda^{I,c}$ (Eq. (38)), fit λ^I against the updated Δ (Section 7.1), mix the updated R^I, Δ^{II} into the running values (Section 7.3), and return to step 1.

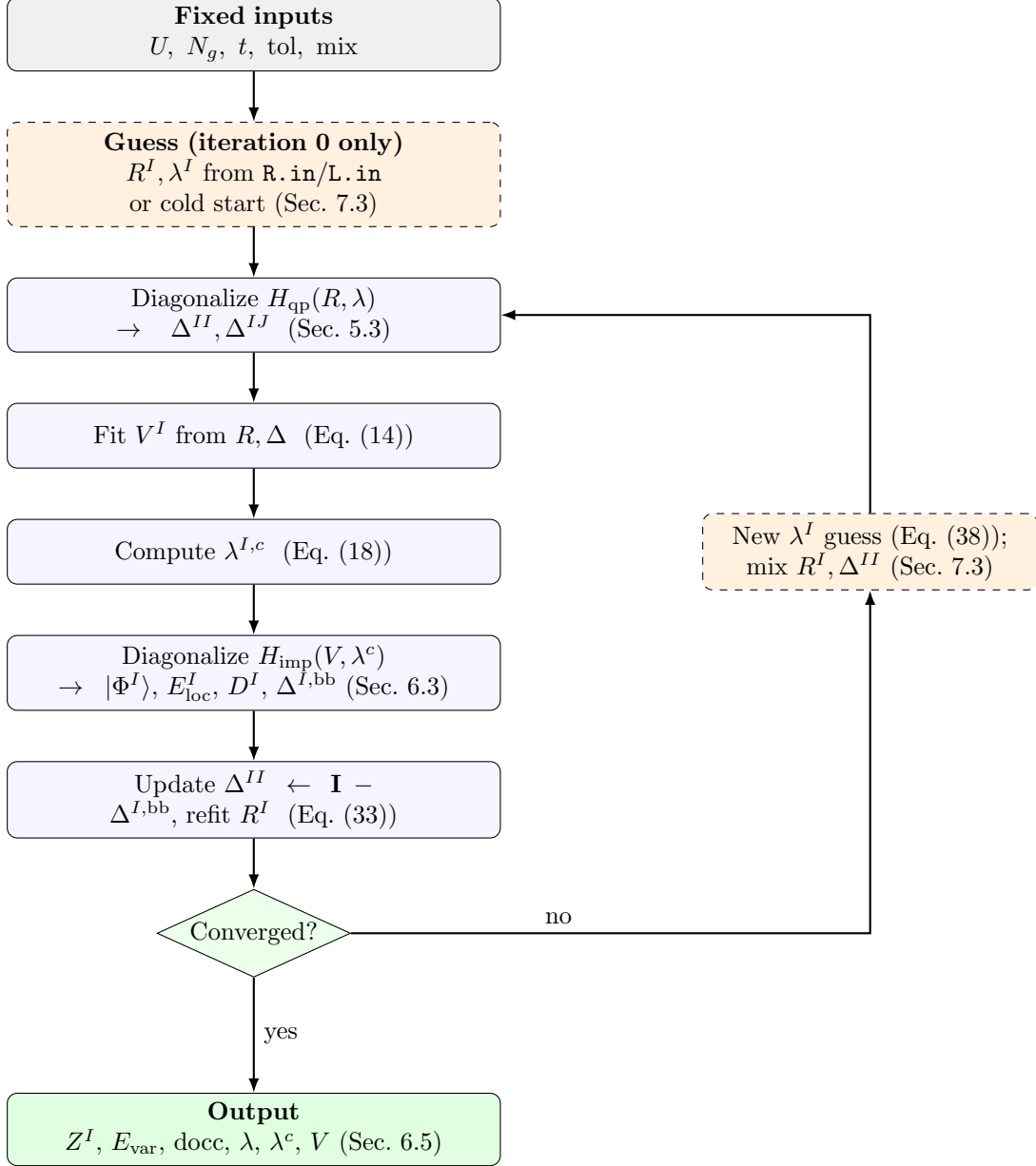


Figure 1: One outer self-consistency iteration. Gray = fixed for the whole run. Orange/dashed = a guess (only ever trusted as a starting point, never as an exact value). Blue = quantities derived, in this order, from the current guess. Green = converged output. The right-hand loop-back box is the *only* place a new λ guess is produced; every other quantity in the main column is a direct evaluation of the equation named inside it.

5 Specialization to the two-site, two-independent-fragment dimer

5.1 Each site is its own fragment; the two fragments need not be equal

Each site is treated as its own impurity/fragment. This is the standard gGA lattice construction applied to a finite two-site “lattice”: lumping the whole dimer into a single impurity would leave no inter-site hopping to renormalize and would make the method vacuous.

Important correction relative to an earlier version of this project. It is tempting to assume that, because the physical Hamiltonian Eq. (1) is symmetric under exchanging sites $0 \leftrightarrow 1$, the two fragments’ variational parameters must be equal ($R^0 = R^1$, $\lambda^0 = \lambda^1$), so that only one impurity problem needs to be solved per iteration. This is *not* true in general. A reference calculation for this same dimer at $U/t = 2$, $N_g = 2$ converges to a *genuinely asymmetric* fixed point, with numerically distinct $R^0 \neq R^1$ and $\lambda^0 \neq \lambda^1$ (even though $|R^0| \approx |R^1|$, their components differ substantially – see Table 1). The physical dimer’s site-exchange symmetry can be *spontaneously broken* at the level of the auxiliary variational parameters, exactly as in a mean-field theory with more than one self-consistent solution. The code therefore **always** treats the two fragments as independent variational objects; the possibility that a converged solution happens to satisfy $R^0 = R^1$ is a property of the solution, never an assumption imposed on the algorithm.

5.2 Quasiparticle Hamiltonian for two independent fragments

Let $R^0, R^1 \in \mathbb{R}^{N_{\text{eff}}}$ (column vectors, since $N_{\text{phys}} = 1$) be the two fragments’ renormalization vectors, and $\lambda^0, \lambda^1 \in \mathbb{R}^{N_{\text{eff}} \times N_{\text{eff}}}$ (real symmetric) their onsite potentials. Because $N_{\text{phys}} = 1$, the fully general multi-orbital quasiparticle Hamiltonian

$$H_{\text{qp}} = \sum_{I \neq J} \sum_{a,b} \left(\sum_{\alpha,\beta} R_{a\alpha}^{I\dagger} t_{IJ}^{\alpha\beta} R_{\beta b}^J - \delta_{IJ} \lambda_{ab}^I \right) d_{Ia}^\dagger d_{Jb} \quad (10)$$

collapses, for our two fragments $I, J \in \{0, 1\}$ and scalar hopping $t_{01} = t_{10} = -t$ (with $t_{IJ}^{\alpha\beta}$ having only one value of α, β), to the explicit $2N_{\text{eff}} \times 2N_{\text{eff}}$ real-space matrix (per spin channel; both spin channels use the same matrix)

$$H_{\text{qp}} = \begin{pmatrix} -\lambda^0 & -t R^0 (R^1)^T \\ -t R^1 (R^0)^T & -\lambda^1 \end{pmatrix} \quad (11)$$

This is the direct generalization of the single-fragment (symmetric) block matrix $H_{\text{qp}} = \begin{bmatrix} -\lambda & -t R R^T \\ -t R R^T & -\lambda \end{bmatrix}$ to the case $R^0 \neq R^1$, $\lambda^0 \neq \lambda^1$; it reduces to the symmetric form exactly when $R^0 = R^1 = R$ and $\lambda^0 = \lambda^1 = \lambda$. Because there are only two real-space sites, H_{qp} is diagonalized directly (no Brillouin-zone integral is needed, unlike the periodic-lattice formulation this is specialized from).

Code: `qp_step2(R0, R1, lam0, lam1, t)` builds exactly Eq. (11) and diagonalizes it with `scipy.linalg.eigh`. `qp_step(R, lam, t)` is the $R^0 = R^1$, $\lambda^0 = \lambda^1$ special case, kept as a separate, faster code path used only in some validation scripts; the production self-consistency loop always calls `qp_step2`.

5.3 Filling rule (half filling)

H_{qp} has $2N_{\text{eff}}$ single-particle levels per spin. Half filling of the physical dimer (one electron per spin, summed over the two sites) fixes the number of *filled quasiparticle levels per spin* to exactly N_{eff} (half of the $2N_{\text{eff}}$ available levels) – *not* to a fixed chemical potential threshold, and *not* simply to

1 level regardless of N_{eff} . Concretely, if $\varepsilon_1 \leq \varepsilon_2 \leq \dots \leq \varepsilon_{2N_{\text{eff}}}$ are the eigenvalues of H_{qp} returned by `eigh` in ascending order, with orthonormal eigenvectors $|n\rangle$, the one-body density matrix of the filled quasiparticle Fermi sea is

$$\rho^{\text{qp}} = \sum_{n=1}^{N_{\text{eff}}} |n\rangle\langle n|. \quad (12)$$

This is a fixed *level-count* rule, independent of the numerical values of the eigenvalues (as long as there is no exact degeneracy straddling the N_{eff} -th/ $(N_{\text{eff}}+1)$ -th level, which does not occur for the solutions studied here). ρ^{qp} is a $2N_{\text{eff}} \times 2N_{\text{eff}}$ projector of rank N_{eff} ; its four $N_{\text{eff}} \times N_{\text{eff}}$ blocks give the three independent quasiparticle 1-RDM blocks needed below:

$$\Delta^{00} = \rho_{[0, N_{\text{eff}}] \times [0, N_{\text{eff}}]}^{\text{qp}}, \quad \Delta^{11} = \rho_{[N_{\text{eff}}, 2N_{\text{eff}}] \times [N_{\text{eff}}, 2N_{\text{eff}}]}^{\text{qp}}, \quad \Delta^{01} = \rho_{[0, N_{\text{eff}}] \times [N_{\text{eff}}, 2N_{\text{eff}}]}^{\text{qp}}. \quad (13)$$

Δ^{00} and Δ^{11} are symmetric (each is a genuine reduced density matrix restricted to one fragment); Δ^{01} is in general *not* symmetric, and $\Delta^{10} = (\Delta^{01})^T$.

Code: inside `qp_step2`, `occ = np.zeros(2*Neff)`; `occ[:Neff] = 1.0` implements exactly the level-count rule of Eq. (12) (`eigh` returns eigenvalues in ascending order, so `occ[:Neff]=1` fills the lowest N_{eff} levels); `Delta00`, `Delta11`, `Delta01` are the three blocks of Eq. (13).

6 Self-consistency equations

The gGA self-consistency loop for two fragments couples the quasiparticle side (Section 5) to an embedding/impurity side through four fitting/update relations, specialized here from the general multi-fragment formulation (Mejuto-Zaera, arXiv:2403.05157, Eqs. 5–11) and validated numerically, block by block, against an independent reference implementation (see Table 1).

6.1 Hybridization V (Eq. 7 of the reference)

For fragment I , the hybridization vector $V^I \in \mathbb{R}^{N_{\text{eff}}}$ is fixed by requiring

$$\sqrt{\Delta^{II}(\mathbf{I} - \Delta^{II})} V^I = -t \Delta^{IJ} R^J, \quad J \neq I, \quad (14)$$

i.e. for our two fragments,

$$\sqrt{\Delta^{00}(\mathbf{I} - \Delta^{00})} V^0 = -t \Delta^{01} R^1, \quad (15)$$

$$\sqrt{\Delta^{11}(\mathbf{I} - \Delta^{11})} V^1 = -t (\Delta^{01})^T R^0. \quad (16)$$

Notice that V^I is fit using the *neighbor* fragment's R^J (not the fragment's own R^I), and the *cross*-fragment block Δ^{IJ} (not Δ^{II}). Getting this backwards (using R^I instead of R^J) is invisible in the symmetric special case $R^0 = R^1$ but produces a completely different, wrong answer once the two fragments differ – this was confirmed empirically while developing the code.

The matrix square root in Eq. (14) is evaluated in the eigenbasis of Δ^{II} : if $\Delta^{II} = U \text{diag}(n_1, \dots, n_{N_{\text{eff}}}) U^T$, Eq. (14) is solved eigenvalue by eigenvalue,

$$V^I = U \text{diag}\left(\frac{1}{\sqrt{n_a(1 - n_a)}}\right) U^T (-t \Delta^{IJ} R^J). \quad (17)$$

Regularization. As the true self-consistent solution is approached, one eigenvalue of Δ^{II} can smoothly go to exactly 0 or 1 (a ghost bath orbital becoming completely empty or completely full); this was observed explicitly for the $U/t = 2$, $N_g = 2$ example, where the smallest eigenvalue of Δ^{00} shrinks from 2.1×10^{-5} to 8×10^{-6} to (numerically) 0 over a few iterations. Exactly at that boundary $1/\sqrt{n_a(1-n_a)}$ diverges. The code guards against this in two independent ways, both applied in `fit_V`:

- the eigenvalues are clipped, $n_a \rightarrow \text{clip}(n_a, \varepsilon, 1 - \varepsilon)$ with $\varepsilon = 10^{-10}$ (deliberately *tiny*: a larger $\varepsilon = 10^{-4}$ was tried and tames the blow-up, but it also biases the well-conditioned $U/t = 10$ solution enough that it converges to the wrong fixed point);
- the *output* components of V^I are clipped to $[-V_{\max}, V_{\max}]$ with $V_{\max} = 10^2$, which only engages exactly when a component would actually blow up and otherwise leaves well-conditioned cases untouched.

Code: `fit_V(R, Delta, Delta_off, t, eps=1e-10, max_V=1e2)`. It is called as `fit_V(R1, Delta00, Delta01, t)` for fragment 0 and `fit_V(R0, Delta11, Delta01.T, t)` for fragment 1, matching Eq. (14) exactly (note the neighbor's R and the transposed off-diagonal block for fragment 1).

6.2 Bath potential λ^c (Eq. 8 of the reference)

The onsite potential λ^I that enters H_{qp} (Eq. (11)) and the bath potential $\lambda^{I,c}$ that enters the impurity Hamiltonian (Section 6.3) are *not* independent variational parameters. They arise as Lagrange multipliers in the underlying constrained variational problem, and stationarity of the Lagrangian with respect to the quasiparticle density matrix $\Delta^{(\text{qp})}$ gives the linear relation

$$\lambda_{ab}^I + \lambda_{ab}^{I,c} = \frac{\partial}{\partial \Delta_{ab}^{II}} \left[R^I \cdot \sqrt{\Delta^{II}(\mathbf{I} - \Delta^{II})} \cdot V^I \right] \Big|_{\text{sym}}, \quad (18)$$

where R^I and V^I are held *fixed* in the derivative (i.e. V^I is *not* re-substituted through Eq. (14) as a function of Δ^{II} before differentiating), and “sym” denotes the standard symmetric-matrix-gradient convention: for $a = b$ the derivative is taken with respect to the single diagonal entry, while for $a \neq b$ it is the *sum* of the two formally independent partial derivatives with respect to Δ_{ab} and Δ_{ba} (equivalently: differentiate along the symmetric perturbation direction $X_{ab} + X_{ba}$, not along X_{ab} alone, where X_{ab} denotes the elementary matrix with a 1 in position (a, b) and 0 elsewhere – denoted X rather than the more common E to avoid any visual clash with the energies E_{qp} , $E^{I,c}$, E_{loc} , E_{var} used throughout this document). Given λ^I (whichever value the outer self-consistency loop currently holds; see Section 7), Eq. (18) is used to *define*

$$\lambda^{I,c} = -\lambda^I + \text{grad}_S(\Delta^{II}, R^I, V^I), \quad \text{grad}_S(\Delta, R, V)_{ab} \equiv \frac{\partial}{\partial \Delta_{ab}} \left[R \cdot \sqrt{\Delta(\mathbf{I} - \Delta)} \cdot V \right] \Big|_{\text{sym}}. \quad (19)$$

Numerical evaluation of grad_S . grad_S is evaluated by *central finite differences* on the scalar function $S(\Delta) \equiv R^T \sqrt{\Delta(\mathbf{I} - \Delta)} V$ (a true scalar because $N_{\text{phys}} = 1$), with step size $h = 10^{-8}$:

$$[\text{grad}_S]_{aa} = \frac{S(\Delta + 2h X_{aa}) - S(\Delta - 2h X_{aa})}{2h}, \quad (20)$$

$$[\text{grad}_S]_{ab} = \frac{S(\Delta + h(X_{ab} + X_{ba})) - S(\Delta - h(X_{ab} + X_{ba}))}{2h}, \quad a \neq b. \quad (21)$$

Perturbing the diagonal entry by $2h$ (rather than h) is deliberate: it matches the off-diagonal convention, which perturbs *both* symmetric partners Δ_{ab} and Δ_{ba} simultaneously by h each, and by linearity this computes $\partial S/\partial\Delta_{ab} + \partial S/\partial\Delta_{ba} = 2\partial S/\partial\Delta_{ab}$ (using $\Delta_{ab} = \Delta_{ba}$ symmetry of the raw partials); the diagonal case has only one partner (itself), so it must be perturbed by $2h$ to pick up the same overall factor of 2. This normalization was confirmed two independent ways: (i) by deriving it from first principles from the underlying Lagrangian’s constraint bookkeeping (the constraint sum $\sum_{ab}$ runs over *all* ordered pairs, and the matrix-basis normalization used in an independent reference implementation’s derivative class divides off-diagonal basis directions by their Frobenius norm-squared, which is exactly the factor-of-two bookkeeping reproduced here), and (ii) empirically: with the $1\times$ (undoubled) diagonal convention, $\lambda^{I,c}$ ’s diagonal entries came out identically zero for a reference case where they should equal λ^I ’s diagonal entries; with the $2\times$ convention they match to 5–6 significant figures.

Regularization of the step size. The naive choice $h = 10^{-5}$ is numerically unstable exactly when Δ has an eigenvalue very close to 0 or 1: the matrix square root $\sqrt{\Delta(\mathbf{I} - \Delta)}$ is then sharply peaked/non-smooth there, and a 10^{-5} perturbation over- or under-shoots the region where the linear (derivative) approximation is valid, giving wrong derivatives by tens of percent. Reducing h to 10^{-8} (the code’s default) removes this instability while remaining far above the floating-point roundoff floor (verified explicitly: at $h = 10^{-9}$ the computed derivative is unchanged from $h = 10^{-8}$ to the precision shown, confirming convergence, not accidental cancellation).

Code: `grad.S(Delta, R, V, h=1e-8); S(Delta, R, V)` implements $S(\Delta) = R^T \sqrt{\Delta(\mathbf{I} - \Delta)}V$ using `scipy.linalg.sqrtn`.

6.3 Impurity (embedding) Hamiltonian

For each fragment I , the embedding Hamiltonian couples the physical, interacting orbital (both spins, operators $c_\uparrow, c_\downarrow$) to N_{eff} non-interacting bath orbitals per spin (operators $d_{a\uparrow}, d_{a\downarrow}$, $a = 1, \dots, N_{\text{eff}}$):

$$H_{\text{imp}}^I = H_{\text{loc}} + \sum_{a=1}^{N_{\text{eff}}} \sum_{\sigma} V_a^I (d_{a\sigma}^\dagger c_\sigma + c_\sigma^\dagger d_{a\sigma}) - \sum_{a,b=1}^{N_{\text{eff}}} \sum_{\sigma} \lambda_{ab}^{I,c} d_{b\sigma} d_{a\sigma}^\dagger, \quad (22)$$

with the local (physical) Hamiltonian, using the *same* U and particle-hole-symmetric $\mu = U/2$ as the physical dimer,

$$H_{\text{loc}} = U n_\uparrow n_\downarrow - \mu (n_\uparrow + n_\downarrow), \quad n_\sigma = c_\sigma^\dagger c_\sigma. \quad (23)$$

Sign of the bath-potential term – a subtlety. Equation (22)’s bath term is written, as it appears in the underlying Lagrangian, as $-\lambda_{ab}^{I,c} d_{b\sigma} d_{a\sigma}^\dagger$ (annihilation operator first). This is *not* the same as $-\lambda_{ab}^{I,c} d_{a\sigma}^\dagger d_{b\sigma}$: using the fermionic anticommutator $\{d_{a\sigma}, d_{b\sigma}^\dagger\} = \delta_{ab}$,

$$d_{b\sigma} d_{a\sigma}^\dagger = \delta_{ab} - d_{a\sigma}^\dagger d_{b\sigma}, \quad (24)$$

so, dropping the physically irrelevant additive constant $-\sum_{a\sigma} \lambda_{aa}^{I,c}$,

$$-\sum_{ab\sigma} \lambda_{ab}^{I,c} d_{b\sigma} d_{a\sigma}^\dagger \hat{=} +\sum_{ab\sigma} \lambda_{ab}^{I,c} d_{a\sigma}^\dagger d_{b\sigma}. \quad (25)$$

The code implements the *right-hand* (normal-ordered, $+$ sign) form of Eq. (25). Implementing the naive left-hand form with a literal $-$ sign in front of $d_a^\dagger d_b$ is a sign error that was present in an

earlier version of the code and produced impurity ground-state energies and 1-RDMs that disagreed with an independent reference implementation by an order of magnitude; it was found and fixed by comparing intermediate quantities (bath–bath and bath–impurity blocks of the impurity 1-RDM) term by term against a reference calculation.

Exact diagonalization via Jordan–Wigner strings. The impurity Hamiltonian, Eq. (22), is diagonalized exactly (dense, full Fock space) using explicit Jordan–Wigner fermion operators. For n_{modes} ordered fermionic modes, the annihilation operator for mode i is

$$c_i = Z^{\otimes i} \otimes c_- \otimes \mathbf{I}^{\otimes (n_{\text{modes}} - i - 1)}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \quad c_- = \begin{pmatrix} 0 & 1 \\ 0 & 0 \end{pmatrix}, \quad (26)$$

acting on the $2^{n_{\text{modes}}}$ -dimensional local Fock space, with the local basis convention $c_-|1\rangle = |0\rangle$, $c_-|0\rangle = 0$ (i.e. $(0,1)^T$ represents the occupied state). The mode ordering used for the impurity problem is

$$0 : c_{\uparrow}, \quad 1 : c_{\downarrow}, \quad 2+2k : d_{k\uparrow}, \quad 3+2k : d_{k\downarrow} \quad (k = 0, \dots, N_{\text{eff}} - 1), \quad (27)$$

giving $n_{\text{modes}} = 2(1 + N_{\text{eff}})$ total modes and confirming Eq. (4)’s Fock dimension $2^{2(1+N_{\text{eff}})} = 4^{1+N_{\text{eff}}} = 4^{2+N_g}$. `exact.dimer.py` uses the identical Jordan–Wigner construction, Eq. (26), for the four physical modes of the full dimer.

Precomputation. All operator products that do not depend on V^I or $\lambda^{I,c}$ (namely $n_{\uparrow}$, $n_{\downarrow}$, H_{loc} , the hybridization bilinears $d_{a\sigma}^\dagger c_\sigma + \text{h.c.}$, and the bath–bath bilinears $d_{a\sigma}^\dagger d_{b\sigma}$) are built *once* per (N_{eff}, U) pair (function `build_impurity_ops`), not rebuilt at every self-consistency iteration; the impurity Hamiltonian for a given iteration is then simply the weighted sum $H_{\text{imp}} = H_{\text{loc}} + \sum_a V_a (\text{hyb bilinear})_a + \sum_{ab} \lambda_{ab}^c (\text{bath bilinear})_{ab}$. This changes the per-iteration cost from $O(N_{\text{eff}}^2 \cdot \text{dim}^3)$ (rebuilding N_{eff}^2 dense operator products of size $\text{dim} \times \text{dim}$ every iteration) to $O(\text{dim}^3)$ (one diagonalization), where $\text{dim} = 4^{1+N_{\text{eff}}}$; it was the dominant bottleneck before being fixed.

Observables. Given the impurity ground state $|\psi\rangle$ (lowest eigenvector of H_{imp}), all needed expectation values are computed via matrix–vector products only (never forming $O(\text{dim} \times \text{dim})$ operator products explicitly, for the same performance reason):

$$\Delta_{ab}^{I,\text{bb}} = \frac{1}{2} \left(\langle \psi | d_{a\uparrow}^\dagger d_{b\uparrow} | \psi \rangle + \langle \psi | d_{a\downarrow}^\dagger d_{b\downarrow} | \psi \rangle \right), \quad (28)$$

$$D_a^I = \frac{1}{2} \left(\langle \psi | d_{a\uparrow}^\dagger c_{\uparrow} | \psi \rangle + \langle \psi | d_{a\downarrow}^\dagger c_{\downarrow} | \psi \rangle \right), \quad (29)$$

$$E_{\text{loc}}^I = \langle \psi | H_{\text{loc}} | \psi \rangle, \quad \text{docc}^I = \langle \psi | n_{\uparrow} n_{\downarrow} | \psi \rangle, \quad n_{\text{phys}}^I = \langle \psi | (n_{\uparrow} + n_{\downarrow}) | \psi \rangle. \quad (30)$$

The spin-average in $\Delta^{I,\text{bb}}$ and D^I assumes a paramagnetic (spin-unpolarized) solution, which is consistent with the fact that V^I and $\lambda^{I,c}$ enter H_{imp} identically for both spin channels.

Code: `build_impurity_ops(Neff, U, mu)` and `impurity_solve(V, lam_c, ops)`.

How D^I is actually evaluated. $D_a^I = \langle \psi | d_{a\sigma}^\dagger c_\sigma | \psi \rangle$ is a *mixed* bath–impurity expectation value, but it is never evaluated by forming the $\text{dim} \times \text{dim}$ operator product $d_{a\sigma}^\dagger c_\sigma$ itself. Instead, since d_a and c are already stored as the elementary annihilation-operator matrices from Eq. (26), each is applied *once* to the ground-state vector $|\psi\rangle$, and the two resulting vectors are combined with a single inner product:

$$|u_a\rangle \equiv d_{a\uparrow} |\psi\rangle, \quad |v\rangle \equiv c_{\uparrow} |\psi\rangle, \quad \langle \psi | d_{a\uparrow}^\dagger c_{\uparrow} | \psi \rangle = \langle u_a | v \rangle, \quad (31)$$

and likewise for the $\downarrow$ spin channel, with D_a^I the real part of the spin average of the two. The dagger on d_a is never applied explicitly: it falls out for free from the conjugate-linearity of the inner product's first argument (`numpy.vdot(x, y) = x.conj() @ y`), so $\langle d_{a\uparrow}\psi|c_{\uparrow}\psi\rangle = \langle\psi|d_{a\uparrow}^\dagger c_{\uparrow}|\psi\rangle$ comes out automatically. The bath-bath block $\Delta_{ab}^{I,bb}$ just above is evaluated the same way, as $\langle u_a|u_b\rangle$ using the same precomputed $|u_a\rangle$ vectors. This reduces the cost of computing every observable in Eq. (32)–above from $O(N_{\text{eff}}^2 \cdot \text{dim}^2)$ (building each mixed operator explicitly, then a matrix-vector product) to $O(N_{\text{eff}} \cdot \text{dim})$ for the N_{eff} vector applications plus $O(N_{\text{eff}}^2)$ cheap inner products – the same precomputation-driven cost reduction already noted above for H_{imp} itself.

Code: the vectors `bu_gs[a]`, `bd_gs[a]`, `cpu_gs`, `cpd_gs` in `impurity_solve` (`gga_dimer.py:591--594`) are exactly $|u_a\rangle$ (both spins) and $|v\rangle$ (both spins) above; `D[a] = 0.5*(np.vdot(bu_gs[a], cpu_gs) + np.vdot(bd_gs[a], cpd_gs)).real` (`gga_dimer.py:603--607`) implements the formula directly.

6.4 Update of R and Δ (Eq. 10 of the reference)

From the impurity ground state's bath-bath and bath-impurity 1-RDM blocks, each fragment's target quasiparticle quantities are updated as

$$\Delta^{II, \text{new}} = \mathbf{I} - \Delta^{I, \text{bb}}, \quad (32)$$

$$\sqrt{\Delta^{II, \text{new}}(\mathbf{I} - \Delta^{II, \text{new}})} R^{I, \text{new}} = D^I. \quad (33)$$

Equation (33) is solved exactly as Eq. (14), in the eigenbasis of $\Delta^{II, \text{new}}$, with the identical regularization (eigenvalue clip $\varepsilon = 10^{-10}$ plus output clip $R_{\text{max}} = 10^2$).

Code: `fit_R(Delta_new, D, eps=1e-10, max_R=1e2)`.

Where Eq. (33) comes from. Equation (33) is not a new, independent equation – it is the mirror image of the V -fit, Eq. (14) (Section 6.1). Recall the coupling term of the Lagrangian, Eq. (5):

$$(V^I)^T \left[\langle \Phi^I | d_I^\dagger c_I | \Phi^I \rangle - \sqrt{\Delta^{II}(\mathbf{I} - \Delta^{II})} R^I \right] + \text{h.c.} \quad (34)$$

V^I sits there purely as a Lagrange multiplier enforcing that the bracket vanish. Varying the Lagrangian with respect to V^I itself does not fix a new value for V^I ; it simply reinstates the constraint the bracket represents:

$$\langle \Phi^I | d_I^\dagger c_I | \Phi^I \rangle = \sqrt{\Delta^{II}(\mathbf{I} - \Delta^{II})} R^I, \quad (35)$$

which is exactly Eq. (33) once the left-hand side is identified with D^I (Section 3.1, “Variation with respect to R^I ” explicitly notes that varying with respect to V^I , rather than R^I , gives back this R -update instead of the V -fit). So Eqs. (14) and (33) are the *same* underlying relation $\sqrt{\Delta(\mathbf{I} - \Delta)} R = D$ -type constraint, solved for different unknowns at different points of the outer iteration: Eq. (14) solves it for V given R, Δ from the qp side, while Eq. (33) solves it for R given D, Δ freshly produced by the impurity solve.

The $\sqrt{\Delta(\mathbf{I} - \Delta)}$ factor is a coherence weight per bath direction: an orbital that is essentially empty ($\Delta_{aa} \rightarrow 0$) or essentially full ($\Delta_{aa} \rightarrow 1$) has no partial occupation left to fluctuate coherently with the physical orbital, so the factor vanishes and that direction cannot support any R component no matter how large D_a is – exactly the singular case the eigenvalue clip and output cap in `fit_R` are guarding against. A half-filled orbital ($\Delta_{aa} = 0.5$) has the maximal factor 0.5 and can carry the most weight. Eq. (33) is thus best read as: the updated renormalization amplitude R^I is the mixed density D^I , rescaled per bath direction by how much coherent single-particle weight is actually available in that direction.

6.5 Variational energy

The total variational energy of the two-site lattice, per the general formula $E_0 = \sum_I E_{\text{imp}}^I + \sum_{I \neq J} \sum_{ab} (R^{I\dagger} t_{IJ} R^J)_{ab} \Delta_{ab}^{(\text{qp})}$ specialized to $N_{\text{phys}} = 1$, $t_{01} = t_{10} = -t$, and summing the ordered pair $(0, 1)$ and $(1, 0)$ contributions (which are equal, since each is the transpose of a scalar), reads

$$E_{\text{var}} = E_{\text{loc}}^0 + E_{\text{loc}}^1 - 4t (R^0)^T \Delta^{01} R^1. \quad (36)$$

The factor of 4 combines: a factor 2 from summing the two ordered fragment pairs $(I, J) = (0, 1)$ and $(1, 0)$ (the physical hopping term already counts the bond once via $c_0^\dagger c_1 + \text{h.c.}$, and the ordered-pair sum over $I \neq J$ reproduces exactly this without double counting), and another factor 2 from summing the two spin channels (which contribute identically under the paramagnetic assumption). $E_{\text{loc}}^0 + E_{\text{loc}}^1$ (not $2E_{\text{loc}}$) is used because the two fragments' impurity ground states are, in general, different.

At $U = 0$, for any N_g , the code reproduces $E_{\text{var}} = -2t$ exactly (the exact, uncorrelated dimer answer), which is used throughout as a hard consistency check on Eq. (36) and on every other formula that feeds into it.

7 Numerical solution of the self-consistency loop

Sections 6.1–6.4 define, for a *given* (λ^0, λ^1) , how to compute $\Delta^{00}, \Delta^{11}, \Delta^{01}$ (Section 5), V^0, V^1 (Section 6.1), $\lambda^{0,c}, \lambda^{1,c}$ (Section 6.2), the impurity solve (Section 6.3), and updated $R^{0,\text{new}}, R^{1,\text{new}}, \Delta^{00,\text{new}}, \Delta^{11,\text{new}}$ (Section 6.4). What remains is: (a) how (λ^0, λ^1) itself is determined at each outer iteration, and (b) how the outer iteration over $(R^0, R^1, \Delta_{\text{target}}^{00}, \Delta_{\text{target}}^{11})$ is driven to a fixed point. This section describes both, together with every numerical safeguard that was found necessary and why.

7.1 Determining λ^0, λ^1 : a coupled, non-unique root-finding problem

Equation (11) shows that fragment 0's and fragment 1's quasiparticle problems are coupled through a single $2N_{\text{eff}} \times 2N_{\text{eff}}$ matrix; consequently, (λ^0, λ^1) cannot be fit to reproduce a target $(\Delta_{\text{target}}^{00}, \Delta_{\text{target}}^{11})$ independently, fragment by fragment. Instead, the code solves the *joint* nonlinear system, over the $2 \times \frac{1}{2}N_{\text{eff}}(N_{\text{eff}}+1)$ independent symmetric-matrix parameters of (λ^0, λ^1) ,

$$\text{qp_step2}(R^0, R^1, \lambda^0, \lambda^1) \Big|_{00,11} = (\Delta_{\text{target}}^{00}, \Delta_{\text{target}}^{11}). \quad (37)$$

This system is badly non-unique. It was found empirically (and is an unavoidable feature of the projector ρ^{qp} 's highly nonlinear dependence on λ) that Eq. (37) has *many* solutions with equally small residual, differing from each other by large amounts. Concretely: starting an off-the-shelf solver (`scipy.optimize.root`, method `hybr`) from an already near-exact guess (residual $\sim 10^{-4}$ – 10^{-7}), the solver was observed to converge to a *different* (λ^0, λ^1) with entries an order of magnitude larger (e.g. diagonal entries jumping from ≈ 2 to ≈ 20 – 40), while still formally satisfying Eq. (37) to a tiny residual ($< 10^{-6}$). Naively calling a generic root-finder on Eq. (37) each outer iteration is therefore *not* safe: it silently jumps between physically inequivalent branches, which manifested as the outer self-consistency loop either drifting slowly away from the correct fixed point over tens of iterations, or oscillating in a stable limit cycle without ever reaching a fixed point.

A continuity-preserving analytic guess. To keep the solver on the physically continuous branch, the starting guess for (λ^0, λ^1) each outer iteration is not the raw previous value, but is computed *analytically* from Eq. (19) solved in the other direction, using the *previous* iteration’s $V^0, V^1, \lambda^{0,c}, \lambda^{1,c}$ together with the *current* target Δ :

$$\lambda^{I,\text{guess}} = \text{grad}_S(\Delta_{\text{target}}^{II}, R^I, V_{\text{prev}}^I) - \lambda_{\text{prev}}^{I,c}. \quad (38)$$

This guess was verified, at a point close to a converged solution, to match the eventual fitted λ^I to 4–5 significant figures *before* any root-finding is performed – i.e. Eq. (38) is already an excellent approximate solution of Eq. (37), and the role of the subsequent numerical solve (below) is only to *locally refine* it, never to search globally.

Switching rule. Early in the outer iteration (far from convergence), $\lambda^{I,\text{guess}}$ of Eq. (38) is used as the starting point x_0 for the fit below. Once the previous iteration’s residual $\|\Delta^{\text{new}} - \Delta_{\text{target}}\|$ has fallen below 1 (an empirical, generous threshold, essentially distinguishing “still far from converged” from “in the local basin”), the starting point is instead taken as the previous iteration’s actual (λ^0, λ^1) . This mirrors the corresponding switching logic in the reference implementation.

Damped, local Newton solve with an analytic Jacobian. Rather than calling a generic (global) root-finder on Eq. (37) from the guess of Eq. (38), the code performs its own *damped, local* Newton iteration, which by construction cannot leave the guess’s basin of attraction (each accepted step strictly decreases the residual norm; no step is ever taken if none of the tried step sizes improves the residual):

- (i) Compute the residual $f(x) = [(\Delta^{00}(x) - \Delta_{\text{target}}^{00})_{\text{tri}}, (\Delta^{11}(x) - \Delta_{\text{target}}^{11})_{\text{tri}}]$, where $x = [\lambda_{\text{tri}}^0, \lambda_{\text{tri}}^1]$ packs the upper-triangular (independent) entries of the two symmetric matrices, and $\text{tri}(\cdot)$ denotes this packing.
- (ii) Compute the analytic Jacobian $J = \partial f / \partial x$ (Section 7.2).
- (iii) Solve $J \delta x = -f(x)$ by least squares (`numpy.linalg.lstsq`).
- (iv) **Trust-region step cap:** if $\|\delta x\| > 1$, rescale $\delta x \rightarrow \delta x / \|\delta x\|$ (i.e. cap the step’s *magnitude* at 1, keeping its direction). This is discussed further below.
- (v) **Backtracking line search:** try $x + \delta x$; if $\|f(x + \delta x)\| < \|f(x)\|$, accept it; otherwise halve the step ($x + \delta x/2, x + \delta x/4, \dots$, up to 30 halvings) until the residual decreases, or give up (leave x unchanged) if no halving helps.
- (vi) Repeat (i)–(v) for up to 30 Newton iterations or until $\|f(x)\| < 10^{-11}$.

If the final (λ^0, λ^1) is non-finite or has an entry larger than $\lambda_{\text{max}} = 10^3$ in absolute value, the fit is rejected and the original guess $(\lambda^{0,\text{guess}}, \lambda^{1,\text{guess}})$ is returned unchanged instead.

Why the trust-region cap in step (iv) is necessary. Even with the damped/backtracking scheme (v), a *single* raw Newton step can still be enormous if J is ill-conditioned, and a backtracking line search can “accept” a step that is still very large (just smaller than the raw one) simply because it happens to reduce the residual by some tiny amount – while being a qualitatively wrong, discontinuous jump. This was observed concretely: as the outer iteration approaches the true solution and one eigenvalue of Δ^{00} approaches 0 (the same near-degeneracy discussed in

Section 6.1), the corresponding quasiparticle level crosses very close to the Fermi surface, and J 's divided-difference structure (Section 7.2) develops a near-zero energy denominator; the resulting raw Newton step can be tens to hundreds of times larger than x itself, and λ^0 's diagonal was observed to jump from ≈ 2 to ≈ -40 over a single outer iteration as a direct result, even though V^0, V^1, R^0, R^1 all remained modest at that same iteration (ruling out a genuine physical divergence). Two alternative fixes were tried and both failed: (a) clipping the eigenvalues that enter fit_V more aggressively, and (b) Lorentzian-regularizing J 's energy denominator directly (replacing $1/\Delta\varepsilon$ by $\Delta\varepsilon/(\Delta\varepsilon^2 + \eta^2)$ for a broadening η). Both did stabilize the problematic $U/t = 2$ case, but at the cost of silently biasing the well-conditioned $U/t = 10$ case enough that it converged, cleanly and without complaint, to a different, wrong fixed point – for *every* value of η tried, down to $\eta = 10^{-6}$. Capping the raw step's *magnitude* directly, leaving J itself untouched, was the only fix found that stabilizes the $U/t = 2$ case while leaving the $U/t = 10$ case completely unaffected (its converged energy did not change at all, to 6 significant figures, with or without the cap, because the cap simply never engages when J is well-conditioned).

Code: `fit_lambda2(R0, R1, t, Delta00_target, Delta11_target, lam0_guess, lam1_guess, max_lam=1e3, max_newton=30, tol=1e-11)`. The single-fragment analogue `fit_lambda` (used only by validation scripts, not by the production loop) is the same algorithm restricted to $R^0 = R^1 = R$, $\lambda^0 = \lambda^1 = \lambda$.

7.2 Analytic Jacobian of the quasiparticle projector

The Jacobian $J = \partial[\Delta_{\text{tri}}^{00}; \Delta_{\text{tri}}^{11}]/\partial[\lambda_{\text{tri}}^0; \lambda_{\text{tri}}^1]$ used in Section 7.1 is computed *exactly* (to numerical precision), not by finite differences, via first-order (Hellmann–Feynman / Daletskii–Krein divided-difference) perturbation theory of the rank- N_{eff} projector ρ^{qp} of Eq. (12). For a Hermitian matrix H_{qp} with eigenpairs $(\varepsilon_n, |n\rangle)$ and a fixed occupation vector $\text{occ}_n \in \{0, 1\}$, and a perturbation $H_{\text{qp}} \rightarrow H_{\text{qp}} + \delta H_{\text{qp}}$, the projector $\rho = \sum_n \text{occ}_n |n\rangle\langle n|$ changes to first order as

$$\delta\rho = \sum_{n,m} F_{nm} \langle n|\delta H_{\text{qp}}|m\rangle |n\rangle\langle m|, \quad F_{nm} = \frac{\text{occ}_n - \text{occ}_m}{\varepsilon_n - \varepsilon_m} \quad (\rightarrow 0 \text{ if } \varepsilon_n = \varepsilon_m). \quad (39)$$

For each independent parameter x_k (one entry of $\text{tri}(\lambda^0)$ or $\text{tri}(\lambda^1)$), $\delta H_{\text{qp}}/\delta x_k$ is exactly -1 (times the same symmetric-basis-matrix convention as Section 6.2) in the corresponding diagonal block of H_{qp} (top-left for λ^0 , bottom-right for λ^1), and zero elsewhere. Equation (39) is evaluated in the eigenbasis of the *current* $H_{\text{qp}}(R^0, R^1, \lambda^0, \lambda^1)$ (i.e. the Jacobian is recomputed at every Newton iteration, at the current x , exactly as required for Newton's method), and the two relevant $N_{\text{eff}} \times N_{\text{eff}}$ diagonal blocks of $\delta\rho$ give the corresponding columns of J .

Validation. This analytic Jacobian was checked against a direct central-finite-difference computation of the same quantity (step $h = 10^{-6}$) and found to agree to within 9×10^{-9} (consistent with $O(h^2)$ truncation error), confirming both the formula and its implementation independently of the analytic derivation.

On the energy-denominator regularization in F_{nm} . As noted in Section 7.1, F_{nm} is left *unregularized* beyond the strict step-function rule $F_{nm} = 0$ when $|\varepsilon_n - \varepsilon_m| < 10^{-10}$ (i.e. exact numerical degeneracy only); any attempt to broaden this rule (Lorentzian or otherwise) to tame the $U/t = 2$ near-degeneracy was found to break the $U/t = 10$ case. The trust-region step cap of Section 7.1 is the mechanism that actually controls the consequences of a transiently large F_{nm} .

Code: `analytic_jacobian2(R0, R1, lam0, lam1, t)` (two fragments); `analytic_jacobian1(R, lam, t)` (single, shared fragment; used only with `fit_lambda`).

7.3 Outer loop: mixing and convergence

With (λ^0, λ^1) determined as in Section 7.1, one full outer iteration proceeds as follows (see also the pseudocode in Section 7.4):

1. Form the guess $\lambda^{I,\text{guess}}$, Eq. (38), for each fragment.
2. Fit (λ^0, λ^1) , Section 7.1, obtaining also the corresponding $\Delta^{00}, \Delta^{11}, \Delta^{01}$.
3. Fit V^0, V^1 , Section 6.1.
4. Compute $\lambda^{0,c}, \lambda^{1,c}$, Eq. (19) (needed both for the impurity Hamiltonian now, and to seed the *next* iteration’s guess, Eq. (38)).
5. Solve the two impurity problems, Section 6.3.
6. Update $\Delta^{00,\text{new}}, \Delta^{11,\text{new}}, R^{0,\text{new}}, R^{1,\text{new}}$, Section 6.4.
7. Compute the residual $\text{diff} = \max(\text{diff}_R, \text{diff}_\Delta)$, with $\text{diff}_R = \max(\|R^{0,\text{new}} - R^0\|, \|R^{1,\text{new}} - R^1\|)$ and similarly for diff_Δ against the current targets.
8. **Update the best-seen state** (see below).
9. If $\text{diff} < \text{tol} = 10^{-8}$, stop.
10. Otherwise, mix: $R^I \leftarrow R^I + \text{mix} \cdot (R^{I,\text{new}} - R^I)$, $\Delta_{\text{target}}^{II} \leftarrow \Delta_{\text{target}}^{II} + \text{mix} \cdot (\Delta^{II,\text{new}} - \Delta_{\text{target}}^{II})$, and continue to the next iteration (up to `max_iter`, default 100).

Mixing parameter. The default is $\text{mix} = 1.0$, i.e. *plain substitution* (no damping) – matching the reference implementation’s default updater. Two more conservative options exist and were both tried and abandoned as the *default*: (a) a Pulay/DIIS mixer (class `DIIS` in the code, unused by `run_gga` but kept for reference/experimentation) and (b) linear under-relaxation with $\text{mix} < 1$. Both were originally introduced to fix an instability observed *before* the trust-region cap of Section 7.1 existed; with that cap in place, plain substitution ($\text{mix} = 1$) was found to reproduce both reference examples (Table 1) correctly, while $\text{mix} < 1$ was, in one case, found to *actively prevent* convergence to the correct answer (the damping fought the Newton solve’s own, already-appropriate, convergence rate). mix remains an adjustable keyword argument of `run_gga` for experimentation.

Best-seen-state safety net. The loop does not simply return whatever state exists when it stops (either by hitting `tol` or by exhausting `max_iter`); instead, at every iteration it records the iterate with the *smallest* diff seen so far (`best_diff`, `best` in the code), and returns *that* one. This guards against a specific failure mode that was observed and partially, but not completely, cured by the trust-region cap: the outer loop can approach the correct fixed point smoothly for several tens of iterations (with diff shrinking steadily) and then, right as an eigenvalue of Δ^{00} crosses very close to 0, take one bad step into a long chaotic transient that eventually resettles into a *different*, incorrect fixed point with an even smaller final residual than the correct transient ever reached. In that situation, “keep the lowest-residual iterate” does not, by itself, guarantee correctness (the wrong fixed point can converge more tightly than the right one’s transient), but it does provide a reproducible, well-defined selection rule, and it was the combination of this safety net *together with* the trust-region cap that ultimately reproduced both reference examples of Table 1 correctly and stably.

Code: the loop described here is `run_gga(Ng, U, t=1.0, max_iter=100, tol=1e-8, mix=1.0, verbose=False, Rg0=None, Rg1=None, lamg0=None, lamg1=None)`, with `Rg1/lamg1` defaulting to `Rg0/lamg0` (a symmetric starting guess) when not given explicitly, and `Rg0/lamg0` defaulting, if nothing is given at all, to $R^0 = (0.9, 0.05, 0.05 \cdot 1.3, 0.05 \cdot 1.6, \dots)$ (a value distinct from a repeated constant in the ghost components, chosen to break the exact ghost-orbital exchange symmetry that would otherwise trap the iteration on a degenerate manifold) and $\lambda^0 = 0$.

7.4 The main self-consistency loop, in pseudocode

```
def run_gga(Ng, U, t=1.0, max_iter=100, tol=1e-8, mix=1.0,
            Rg0=None, Rg1=None, lamg0=None, lamg1=None):
    Neff = 1 + Ng
    mu = U / 2.0
    ops = build_impurity_ops(Neff, U, mu) # Sec. 3.4, precomputed once

    R0, R1 = seed_or_default(Rg0, Rg1) # Sec. 4.2
    lam0, lam1 = seed_or_default(lamg0, lamg1)

    Delta00_target, Delta11_target, Delta01 = qp_step2(R0, R1, lam0, lam1, t) # Eq. (3)
    - (5)
    V0 = fit_V(R1, Delta00_target, Delta01, t) # Eq. (7)
    V1 = fit_V(R0, Delta11_target, Delta01.T, t)
    lamc0 = grad_S(Delta00_target, R0, V0) - lam0 # Eq. (10)
    lamc1 = grad_S(Delta11_target, R1, V1) - lam1
    ddelta_prev = 10.0

    for it in range(max_iter):
        guess0 = grad_S(Delta00_target, R0, V0) - lamc0 # Eq. (14)
        guess1 = grad_S(Delta11_target, R1, V1) - lamc1
        x0 = (guess0, guess1) if ddelta_prev > 1.0 else (lam0, lam1)

        lam0, lam1, Delta00, Delta11, Delta01 = \
            fit_lambda2(R0, R1, t, Delta00_target, Delta11_target, *x0) # Sec. 4.1

        V0 = fit_V(R1, Delta00, Delta01, t)
        V1 = fit_V(R0, Delta11, Delta01.T, t)
        lamc0 = grad_S(Delta00, R0, V0) - lam0
        lamc1 = grad_S(Delta11, R1, V1) - lam1

        Delta00_bb, D0, Eloc0, docc0, nphys0 = impurity_solve(V0, lamc0, ops) # Sec. 3.4
        Delta11_bb, D1, Eloc1, docc1, nphys1 = impurity_solve(V1, lamc1, ops)

        Delta00_new = I - Delta00_bb # Eq. (11)
        Delta11_new = I - Delta11_bb
        R0_new = fit_R(Delta00_new, D0) # Eq. (12)
        R1_new = fit_R(Delta11_new, D1)

        diff_R = max(|R0_new-R0|, |R1_new-R1|)
        diff_D = max(|Delta00_new-Delta00_target|, |Delta11_new-Delta11_target|)
        diff = max(diff_R, diff_D)

        record_if_best(diff, current_state) # Sec. 4.3
        if diff < tol: break
```

```

R0 += mix * (R0_new - R0) # Sec. 4.3, mix=1.0 by default
R1 += mix * (R1_new - R1)
Delta00_target += mix * (Delta00_new - Delta00_target)
Delta11_target += mix * (Delta11_new - Delta11_target)
ddelta_prev = diff_D

return best_state # E_var via Eq. (13), Z0=|R0|^2, Z1=|R1|^2, docc, n_phys

```

7.5 An abandoned fix: gauge-fixing

The code contains a function `fix_gauge(R, lam, Delta)` implementing a genuine, exact gauge freedom of the single-fragment formulation: any orthogonal $N_{\text{eff}} \times N_{\text{eff}}$ rotation Q of one fragment's ghost-orbital basis, applied as $R \rightarrow RQ$, $\lambda \rightarrow Q^T \lambda Q$, $\Delta \rightarrow Q^T \Delta Q$, leaves every physical prediction (energies, Δ , impurity solve) exactly invariant, since H_{qp} only depends on R through $R \otimes R$ and on λ only through its role inside the same rotated basis. `fix_gauge` pins this freedom down via a Householder reflection rotating R onto the canonical direction $R \rightarrow (|R|, 0, \dots, 0)$, mirroring the reference implementation's SVD-based `FixGauge` (which is itself *disabled by default* in that implementation, `gg_fix_gauge=false`).

This function is **not called** by the production loop (`run_gga`). It was introduced to cure a persistent, small-amplitude oscillation of R (while Δ and the energy converged cleanly) observed with an earlier, buggier version of `fit_lambda2`, and it did cure that oscillation for the symmetric $U/t = 10$ case. However, once `fit_lambda2` was fixed to use the damped, local Newton solve of Section 7.1 (which, unlike the earlier buggy version, stays in the guess's basin by construction and has no reason to drift in gauge space), applying `fix_gauge` was found to be actively *harmful*: for the identical $U/t = 2$ seed that otherwise converges correctly to $E_{\text{var}} = -3.1257$ (Table 1), enabling `fix_gauge` made the loop converge cleanly and stably to a *different*, unphysical fixed point ($E_{\text{var}} \approx -2.04$, a fully decoupled/trivial solution). `fix_gauge` is kept in the code, documented, and available for experimentation, but is not part of the default algorithm.

8 Observables reported by the code

- **Quasiparticle weight** $Z^I = \|R^I\|^2 = (R^I)^T R^I$, reported per fragment as Z0, Z1.
- **Variational energy** E_{var} , Eq. (36).
- **Double occupancy** $\text{docc} = \frac{1}{2}(\text{docc}^0 + \text{docc}^1)$, the fragment-averaged impurity double occupancy (Section 6.3), directly comparable to `exact_dimer.py`'s `docc/site` column.
- **Physical filling** $n_{\text{phys}} = \frac{1}{2}(n_{\text{phys}}^0 + n_{\text{phys}}^1)$.
- **Converged parameters** $R^0, R^1, \lambda^0, \lambda^1, \lambda^{0,c}, \lambda^{1,c}, V^0, V^1$, all returned in the result dictionary and, from the command line, written to `<prefix>.R.dat`, `<prefix>_lambda.dat`, `<prefix>_lambda_c.dat` (one row-block per fragment: atom 0 then atom 1).
- **Spectral function** $A(\omega)$ (Section 9), computed only if `--omega`, `--eta`, `--domega` are given, written to `A_omega.txt`.

9 Post-processing: quasiparticle Green's function and spectral function

Once the outer loop of Section 7 has converged, the same converged $H_{\text{qp}}(R^0, R^1, \lambda^0, \lambda^1)$ (Eq. (11)) can optionally be reused, as a cheap post-processing step, to evaluate the one-body quasiparticle Green's function and, from it, the physical spectral function of the dimer. This is a diagnostic/-plotting step only: it does not feed back into the self-consistency loop.

Quasiparticle Green's function. On a uniform real-frequency grid running from $-\omega_{\text{max}}$ to $+\omega_{\text{max}}$ in steps of $d\omega$ (all three, ω_{max} , the broadening η , and $d\omega$, are supplied by the user – Section 10), the $2N_{\text{eff}} \times 2N_{\text{eff}}$ retarded quasiparticle Green's function is

$$G(\omega) = [(\omega + i\eta) \mathbf{I} - H_{\text{qp}}]^{-1}, \quad (40)$$

with H_{qp} the same fixed, converged matrix of Eq. (11) (the same one for both spin channels, since H_{qp} does not carry a spin index) and $\mathbf{I}$ here the $2N_{\text{eff}} \times 2N_{\text{eff}}$ identity.

Transformation to the physical basis. The physical creation operator of fragment I is related to the qp/bath operators by $c_I^\dagger \rightarrow \sum_a R_a^I d_{Ia}^\dagger$ (Eq. 14 of `NotesOngGut.pdf`, already used in Section 3). Collecting both fragments' R vectors into a single block-diagonal $2N_{\text{eff}} \times 2$ matrix,

$$R_{\text{mat}} = \begin{pmatrix} R^0 & 0 \\ 0 & R^1 \end{pmatrix} \in \mathbb{R}^{2N_{\text{eff}} \times 2}, \quad (41)$$

the 2×2 physical-basis (site-resolved) Green's function is the congruence transform

$$G_{\text{phys}}(\omega) = R_{\text{mat}}^T G(\omega) R_{\text{mat}}, \quad (42)$$

whose diagonal entries are the site-0/site-1 physical Green's functions and whose off-diagonal entries are the inter-site physical Green's function mediated by the qp sector.

Spectral function. Tracing Eq. (42) sums the two sites' contributions; the total physical spectral function is

$$A(\omega) = -\frac{2}{\pi} \text{Im Tr}[G_{\text{phys}}(\omega)] \quad (43)$$

The factor of 2 accounts for both spin channels: H_{qp} , and hence $G(\omega)$ and $G_{\text{phys}}(\omega)$, is identical for $\uparrow$ and $\downarrow$ (the paramagnetic solutions this code targets never distinguish them), so the total spectral weight is twice the single-spin trace computed from Eq. (40)–(42).

Sum rule (validation). For a normalized single-particle Green's function, $\int_{-\infty}^{\infty} A(\omega) d\omega$ must equal the total number of physical (site, spin) degrees of freedom, i.e. 2 sites $\times$ 2 spins = 4. Numerically integrating $A(\omega)$ from `examples/Ng2.U10` ($U/t = 10$, $N_g = 2$) over $\omega \in [-15, 15]$ with $\eta = 0.05$, $d\omega = 0.05$ gives $\int A d\omega \approx 3.96$, within the expected error from the finite frequency cutoff and the Lorentzian broadening tails that extend (slightly) past $\pm\omega_{\text{max}}$ – confirming both the overall normalization and the factor of 2 in Eq. (43).

Code: `compute_spectral_function(R0, R1, lam0, lam1, t, omega_max, eta, domega)` builds H_{qp} and R_{mat} once, then loops over the frequency grid (built with `numpy.arange` padded by half a step so the closed endpoint $+\omega_{\text{max}}$ survives floating-point round-off) evaluating Eqs. (40)–(43) at each point; `save_spectral_function` writes the two-column result (`omega, A(omega)`) to `A.omega.txt`.

10 Command-line interface and input file formats

```
python3 gga_dimer.py --U <U/t> --Ng <even integer>
                        [--t <hopping, default 1.0>]
                        [--max-iter <default 100>]
                        [--quiet]
                        [--R-file <path, default R.in>]
                        [--lam-file <path, default L.in>]
                        [--omega <omega_max> --eta <eta> --domega <step>]
```

Called with no `--U/--Ng`, the script instead prints a default sweep over $N_g \in \{0, 2, 4\}$ and $U/t \in \{0, 1, 2, 4, 8\}$, using the default cold-start seed for every point (no external guess files are read in this mode).

`--omega`, `--eta`, `--domega`. Optional, and only meaningful together with `--U/--Ng`; the three must be given together (any one given without the other two is a CLI error). When given, after the self-consistency loop converges the code evaluates the spectral function of Section 9 on the grid `-omega` to `+omega` in steps of `domega`, with broadening `eta`, and writes `A_omega.txt` in the current directory. None of the three has a default: the user must supply all three explicitly, since there is no physically motivated default frequency window, broadening, or resolution – these depend entirely on the energy scales (U , t , the converged λ , λ^c) of the particular run.

R.in. Either *one* row of N_{eff} numbers (used as a symmetric guess, $R^0 = R^1 = \text{that row}$) or *two* rows of N_{eff} numbers each (fragment 0’s row, then fragment 1’s row), free-format whitespace- or comma-separated.

L.in. Either N_{eff} rows of N_{eff} numbers (one symmetric matrix, used for both fragments) or $2N_{\text{eff}}$ rows of N_{eff} numbers (fragment 0’s $N_{\text{eff}} \times N_{\text{eff}}$ block, stacked on top of fragment 1’s block), free-format.

Both files are read automatically if present in the working directory (or at the path given by `--R-file/--lam-file`); if absent, the default cold-start seed of Section 7.3 is used. Every run, successful or not, writes its final (R, λ, λ^c) back out in exactly this stacked, two-row-block format (`save_results`), so a converged run’s output can be fed back in directly as the next run’s guess.

11 Worked examples and current validation status

Two examples, seeded directly from an independent reference implementation’s own converged (or near-converged) iterates, are kept under `examples/` and are the primary validation of everything in this document:

Example	N_g	U/t	seed type	this code’s E_{var}/t	reference E_{var}/t
<code>examples/Ng2_U10</code>	2	10	symmetric ($R^0=R^1$)	−10.101314	−10.1013
<code>examples/Ng2_U2</code>	2	2	asymmetric ($R^0 \neq R^1$)	−3.125656	−3.12566

Table 1: The two seeded reference examples and the agreement obtained with the algorithm of Section 7, matching the independent reference implementation to 5–6 significant figures in both cases.

Known open limitation: cold-start reliability. The fixed-point map defined by Sections 6–7 has, at fixed (N_g, U) , more than one solution (an expected feature of a mean-field-like variational theory, directly analogous to a Hartree–Fock calculation having multiple self-consistent solutions of which only one is the true minimum). Starting from the default, generic cold-start seed (Section 7.3) does *not* reliably converge to the lowest-energy, physically correct solution for every (N_g, U) ; it can land on trivial (decoupled, $R \rightarrow \text{small}$) or otherwise spurious branches. Both examples in Table 1 required seeding `R.in/L.in` directly from the reference implementation’s own trajectory to reliably reproduce the correct answer. Making the cold start itself robust (e.g. via a systematic multi-start strategy, or continuation in U from the exactly solvable $U = 0$ point) is identified as future work and is not yet implemented.

12 References

1. N. Lanatà *et al.*, “Phase Diagram and Electronic Structure of Praseodymium and Plutonium,” *Phys. Rev. X* **5**, 011008 (2015); and N. Lanatà *et al.*, *Phys. Rev. X* **7**, 031013 (2017) – original ghost Gutzwiller approximation.
2. F. Mejuto-Zaera and M. Fabrizio, “Efficient computational screening of strongly correlated materials: Multi-orbital phenomenology within the ghost Gutzwiller approximation,” arXiv:2305.03329 – procedural formulation for periodic lattices (source of the general H_{qp} , filling, and update-equation structure specialized in Section 5).
3. F. Mejuto-Zaera, “Quantum embedding for molecules using auxiliary particles – the ghost Gutzwiller ansatz,” *Faraday Discuss.* **254**, 653 (2024), arXiv:2403.05157 – finite/molecular (non-periodic) formulation; the direct source of Eqs. (14)–(36) (Eqs. 7–11 of that paper) as specialized here to the two-site, two-independent-fragment dimer.
4. C. Mejuto-Zaera, “Notes on (ghost) Gutzwiller” (unpublished notes, `NotesOngGut.pdf`) – the source of the Lagrangian derivation underlying Eq. (18) (its Eq. 29) and of the impurity Hamiltonian’s exact operator ordering, Eq. (22) (its Eq. 30), including the sign subtlety of Eq. (25).
5. Independent reference implementation (C++, `gGut` package): used throughout as ground truth for validating every equation and numerical convention in this document, via full iteration-by-iteration logs at $U/t = 2$ and $U/t = 10$ (files `2.0.out`, `10.0.out`) and via direct inspection of the relevant source files (`ggutzwiller.c++`, `lambd_opt.c++`, `delta_derivative.c++`).